



CS222: Data Structures and Algorithms
Spring Semester (May - August 2026)
In-Class Assignment 2 (Linked Lists)

STATEMENT ABOUT ACADEMIC HONESTY AND INTEGRITY

Academic honesty and integrity are very important at Ashesi and central to achieving our mission: to train a new generation of ethical and entrepreneurial leaders in Africa, cultivating within our students critical thinking skills, concern for others, and the courage it will take to transform a continent. As this mission serves as our moral compass, we recommend you take it seriously in this course, with no exceptions.

Ashesi therefore does not condone any form of academic dishonesty, including plagiarism, cheating on tests and assessments, and other such practices. Ashesi requires students to complete their own assignments and produce their own academic work, unless given a group assignment.

As stated in Ashesi's student handbook, Section 7.4:

"Academic dishonesty includes plagiarism, unauthorised exchange of information or use of material during an examination, unauthorised transfer of information or completed work among students, use of the same paper in more than one course, unauthorised collaboration on assignments, and other unethical behaviour. Disciplinary action will be taken against perpetrators of academic dishonesty."

All forms of academic dishonesty are considered misconduct under Ashesi Student Rules and Regulations, including the unauthorised use **of AI tools**. Students who commit academic dishonesty will be brought before the Ashesi Judicial Committee, and such acts of academic dishonesty will have serious consequences for their academic records.

INSTRUCTIONS:

1. This is an **individual in-class assignment**. So, every student must work independently in the lab class and submit their work at the end of the class.
2. This lab assignment contains *two* questions. You are required to solve both.
3. **Important Note:** This assignment is released before the actual lab session. You are expected to prepare at least a half-page of handwritten design notes of the programs that you can use to guide your implementation during the lab session. These notes should be high-level design notes, **not a detailed implementation of any class or method**. You will upload a picture of your design page to Canvas **before** the lab session and bring your paper to the lab session.
4. You are **not allowed to use any online resources** during the implementation of your programs. We will distribute a Java cheat sheet (a document containing major syntactic elements, Java features, and libraries) if it is helpful.
5. The weightage for the questions is distributed as 40% for the first question and 60% for the second. The total assignment will be evaluated for 100 points.
6. Your solution program should contain comments explaining the program statements that involve important computations.
7. Your submission will be a zip file containing your original programs (.java files), screenshots of the program execution (more information will be provided on this), and the scanned/photographed handwritten design notes.
8. **Marking Criteria:**
 - Points for each question are awarded as follows.
 - ✓ Initial design notes: 10 points
 - ✓ Solution program: 15 points
 - ✓ Comments on program statements: 5 points
 - ✓ Correct execution and screenshots: 20 points

PROBLEM 1: Building a Doubly Linked List (40% weighting).

This exercise challenges you to implement a reusable, generic Doubly Linked List structure in Java. That means you must build your own custom doubly linked list, for example, `MyDoublyLinkedList`, that mimics the behaviour of a standard Doubly Linked List.

Requirements:

- **Generic Node Class (`Node<T>`):**
 - Create a generic node class that can store any object type `T`
 - Attributes:
 - `T data`: The payload of the node.
 - `Node<T> next`: Reference to the next node (initially null).
 - `Node<T> prev`: Reference to the previous node (initially null).
 - Implement constructors, getters and setters as needed.
- **Generic Doubly Linked List (`MyDoublyLinkedList<T>`):**

- Create a class `MyDoublyLinkedList<T>` that manages the nodes.
- Attributes:
 - `Node<T> head`: Points to the first node in the list.
 - `Node<T> tail`: Points to the last node in the list.
- **Implement the following methods:**
 - `public void add(T element)`: Adds a new node containing element to the end of the list (updating head and tail pointers accordingly).
 - `public boolean contains(T element)`: Returns true if the element exists in the list.
 - `public boolean remove(T element)`: Searches for the element (using `.equals()`), removes its node from the list, corrects surrounding pointers, and returns true if successful.
 - `public T getFirst()`: Returns the node at the head of the list.
 - `public T getLast()`: Returns the element at the tail.
 - `public void displayForward()`: Prints all elements in order from oldest (head) to newest (tail).
 - `public void displayBackward()`: Prints all elements in reverse order from newest (tail) to oldest (head).

PROBLEM 2: Music Playlist Manager with Continuous Play (60% weighting).

In this problem, you will apply the generic `MyDoublyLinkedList<T>` you built in Problem 1 to create a music playlist application.

Requirements:

- The Song class
 - Design a Song class with attributes: `title (String)`, `artist (String)`, and `duration (double, in minutes)`.
 - Override the `equals(Object obj)` method so that songs are compared and matched by their title.
 - Implement a `toString()` method to output the song's details cleanly.
- **The PlaylistManager Class:**
 - This class manages the playlist using your custom `MyDoublyLinkedList<Song>`.
 - **Attributes:**

- `MyDoublyLinkedList<Song> playlist`: The underlying storage.
- `Node<Song> currentPlaying`: Tracks the song currently playing.
- `boolean isContinuousPlay`: Tracks whether continuous loop mode is active (default is false).
- **Implement the following helper methods:**
 - `public void addSong(Song song)`: Adds a song to the playlist. If it is the first song, set it as `currentPlaying`.
 - `public void removeSong(Song song)`: Removes a song from the playlist. If the removed song was currently playing, safely advance the pointer to another song (or null if empty).
 - `public void toggleContinuousPlay()`: Flips the state of the continuous play mode.
 - `public void playNext()`: Advances `currentPlaying` to the next song.
 - Normal Mode: If you reach the end of the playlist, `currentPlaying` becomes null.
 - Continuous Play Mode: If you reach the end of the playlist, wrap around so `currentPlaying` points back to the head of the list.
 - `public void playPrevious()`: Moves `currentPlaying` to the previous song.
 - Normal Mode: If you reach the beginning of the list, `currentPlaying` becomes null.
 - Continuous Play Mode: If you are at the beginning of the list, wrap around so `currentPlaying` points to the tail of the list.
- **In your main method (driver program), instantiate a `PlaylistManager` and perform the following sequence:**
 - Add 3 songs to your playlist.
 - Print the playlist forward and backwards.
 - Play the first song and step forward twice to show sequential navigation.
 - Enable Continuous Play, step forward again to demonstrate wrapping around to the first song.
 - Remove one of the songs from the playlist, print the updated list, and verify that navigation still works.